Backend Development and Jury Preparation Handbook

Exam Scheduling and Management System

Gautam Buddha University Examination Operations

Prepared from the current PHP application implementation

How to Use This Handbook

This handbook explains how the backend receives a request, protects it, applies examination rules, changes the database and returns a response. It is written for project demonstration and jury preparation. The implementation is a custom PHP application using a single public entry point, server-rendered views, PDO with MySQL, domain services and integration tests.

- Study Sections 1 to 4 for architecture and request flow.
- Study Sections 5 to 9 for authentication, business services, imports and exports.
- Use Sections 10 to 12 for testing, deployment, troubleshooting and live demonstration.
- Revise the question bank and the final two-minute explanation immediately before the jury.

Contents

1. Backend Overview

2. Technology Stack and Project Structure

3. Request Lifecycle and Routing

4. Foundation Components

5. Authentication Authorization and Security

6. Validation and Data Access

7. Automatic Scheduling Backend

8. Seating Invigilation and Attendance

9. Bulk Import and Export

10. Error Handling Audit and Deletion Safety

11. Testing Strategy

12. Deployment Performance and Maintenance

13. Backend Jury Question Bank

14. Appendix A Route and Module Demonstration

15. Appendix B Presentation Script and Checklist

1 Backend Overview

The backend is responsible for much more than database CRUD. It authenticates users, applies role permissions, validates academic and operational rules, stages bulk uploads, generates date sheets and seating plans inside transactions, records audit history and exports official results. The design keeps HTTP concerns in the application layer and complex rules in focused domain services.

Area	Current implementation
Language	PHP with strict types
Web server	Apache with mod_rewrite and security headers
Database access	PDO MySQL with native prepared statements
Rendering	Server-side PHP views and shared layout
Client enhancement	Vanilla JavaScript for interactive scheduling and corrections
Session model	Secure PHP session cookie with timeout and CSRF token
Exports	CSV and native XLSX Office Open XML
Testing	PHP assertion tests plus temporary-database integration tests

Architectural style

```
Browser -> Apache rewrite -> public/index.php -> Application dispatch
        -> authorization -> handler -> domain service -> PDO/MySQL
        -> View or Response -> Browser
```

2 Technology Stack and Project Structure

Folder	Backend responsibility
public	Document root, front controller, static assets and rewrite rules
bootstrap	Autoloading, environment loading and application creation
config	Application and database configuration
app/Foundation	Boot, routing, database and session lifecycle
app/Http	Request and response abstractions
app/Auth	Authentication and role permissions
app/Academic	Roll parsing, student access and batch progression
app/Exams	Cycles, scheduling, review, publication and deletion
app/Import	CSV/XLSX staging, validation and commit services

Folder	Backend responsibility
app/Seating and app/Rooms	Seat layout and allocation algorithms
app/Export	XLSX creation
resources/views	Server-rendered page templates
database	Canonical schema and baseline seed
tests	Unit-like and integration regression tests

Major backend components

Entry and bootstrap

Component	Responsibility
public/index.php	Accepts every rewritten request, captures it and sends the application response.
bootstrap/app.php	Loads helpers, registers the App namespace autoloader, loads environment values and boots the application.

Configuration

Component	Responsibility
config/app.php	Application name, environment, debug mode, base URL, timezone and session timeout.
config/database.php	MySQL host, port, database, username, password and utf8mb4 charset.

Foundation

Component	Responsibility
Application	Central front controller, route dispatcher, authorization gate and request workflow coordinator.
Environment	Reads a simple key-value .env file into the runtime environment.
Database	Creates and reuses a strict PDO connection and exposes a health check.
Session	Secure cookie settings, timeout, flash data, CSRF token, regeneration and invalidation.

HTTP

Component	Responsibility
Request	Normalizes method and route path and exposes GET or POST input.
Response	Produces HTML, JSON, redirects, CSV and XLSX downloads with correct headers.

Views

Component	Responsibility
View	Renders PHP templates with extracted data and wraps protected pages in the shared layout.

Authentication

Component	Responsibility
Auth	Verifies active users with password_verify, regenerates sessions and updates last login.

Authorization

Component	Responsibility
RolePolicy	Maps eight roles to explicit application permissions.

Academic

Component	Responsibility
RollNumberParser	Validates and normalizes institutional roll-number structure.
BatchProgression	Determines the active semester from programme duration, batch and academic session.
StudentRepository	Encapsulates student persistence and lookup logic.

Examination

Component	Responsibility
ExamService	Creates exam cycles, schedules papers and resolves programme and batch registrations.
SchedulingValidator	Validates academic scope, written papers, calendar, shifts, gaps and capacity.

Component	Responsibility
AutomaticScheduler	Generates transactional branch and batch aware examination placements.
ScheduleReviewService	Locks papers, validates manual moves and prepares regeneration.
SchedulePublicationService	Runs final checks and controls approval and publication states.
WrittenPaperPolicy	Excludes laboratory and practical subjects from written schedules.
CourseDeletionService	Deletes curriculum mappings only after dependency checks.
ExamCycleDeletionService	Provides preview, fingerprint and two-step deletion for complete cycles.

Seating

Component	Responsibility
SeatingAllocator	Generates versioned round-robin seating and validates capacity and duplicates.

Rooms

Component	Responsibility
RoomSeatGenerator	Creates visual room-seat coordinates and blocked-seat state.

Imports

Component	Responsibility
TabularReader	Reads CSV and XLSX without requiring a spreadsheet application.
Import services	Stage, validate and commit students, faculty, rooms, programmes, courses and date sheets.

Exports

Component	Responsibility
XlsxExporter	Builds valid Office Open XML workbooks for date-sheet exports.

3 Request Lifecycle and Routing

1. Apache serves a real public file directly or rewrites the route to public/index.php.

2. The bootstrap registers autoloading, loads .env, creates Application and calls boot.

3. Request capture removes the script directory and normalizes the method and path.

4. Application handles health and login routes before protected routes.

5. Unauthenticated users are redirected to login; authenticated users pass a permission check.

6. The dispatcher matches a fixed path or a regular expression with numeric identifiers.

7. A handler validates input and calls a service or database query.

8. The backend returns HTML, JSON, redirect, CSV or XLSX through Response.

9. Unhandled exceptions become a safe 500 response in production.

HTTP method conventions

Method	Use	Example
GET	Read or render a page	/date-sheets/5/automatic
POST	Create, update, publish or delete	/date-sheets/5/automatic/generate
GET download	Generate a read-only export	/date-sheets/5/export.xlsx
JSON POST	Live validation preview	/date-sheets/5/automatic/preview

4 Foundation Components

Database connection

The Database class creates the PDO object only when first needed and reuses it. Native prepares are enforced, errors throw exceptions and associative rows avoid numeric duplicates.

```
PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION
PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC
PDO::ATTR_EMULATE_PREPARES => false
```

Request and response

Request offers the normalized HTTP method, path and combined form/query input. Response centralizes status, headers and body so controllers do not directly echo or call header throughout the application.

View rendering

View resolves templates under resources/views, extracts only supplied data without overwriting existing variables, captures output buffering and wraps protected pages in the main layout. Authentication and error pages render without that layout.

Configuration

Variable	Purpose	Production expectation
APP_ENV	Runtime environment	production

Variable	Purpose	Production expectation
APP_DEBUG	Detailed error visibility	false
APP_URL	Absolute URL generation	Exact HTTPS domain and public path
APP_TIMEZONE	Date and time calculations	Asia/Kolkata
SESSION_TIMEOUT_MINUTES	Inactivity timeout	Institution-approved value
DB_HOST and DB_PORT	Database server	Hosting provider values
DB_DATABASE	Selected application database	Exact created database name
DB_USERNAME and DB_PASSWORD	Database authentication	Never commit to Git

5 Authentication Authorization and Security

Login sequence

1. Validate CSRF token and temporary login throttle.

2. Load the user and role with a prepared statement.

3. Require active status and verify password hash.

4. Regenerate the session identifier.

5. Store only the safe identity and role payload in the session.

6. Update last login and redirect to dashboard.

Role permission matrix

Role	Permissions	Operational purpose
admin	All permissions	System configuration, users, masters, operations and audit
examination_controller	Dashboard, exams, seating, attendance, reports	Controls examination operations
academic_coordinator	Dashboard, academic, students, reports	Maintains programmes, batches, curriculum and students
seating_coordinator	Dashboard, rooms, seating, reports	Maintains layouts and seating
invigilation_coordinator	Dashboard, faculty, invigilation, attendance, reports	Manages duties and replacements
faculty	Dashboard	Restricted authenticated access
auditor	Dashboard, reports, audit	Read-only compliance review
viewer	Dashboard, reports	Authorized read-only reporting

Security controls

Threat	Backend control
SQL injection	Prepared statements with bound parameters and native prepares
Cross-site request forgery	Session CSRF token on every state-changing form
Session fixation	Session ID regeneration after successful login
Session theft	HttpOnly, SameSite and HTTPS Secure cookie attributes
Cross-site scripting	Central e helper for HTML escaping
Brute-force login	Five-attempt counter and sixty-second session pause
Clickjacking	X-Frame-Options SAMEORIGIN
MIME confusion	X-Content-Type-Options nosniff
Sensitive file access	Public document root and Apache denial for hidden env and Git files
Information leakage	Generic production 500 page and APP_DEBUG false

6 Validation and Data Access

Three validation layers

Layer	Responsibility	Example
Request	Required fields, formats and CSRF	Programme code or selected room IDs
Domain service	Cross-record business rules	Batch belongs to programme and paper fits shift
Database	Final structural integrity	Foreign keys and unique seat assignment

Transaction pattern

```
beginTransaction
try
    validate current state
    write parent and child records
    write audit record
    commit
catch Throwable
    rollback
    rethrow or flash a safe error
```

Transactions are used for imports, schedule generation, seating allocation, attendance updates and dependency-aware deletion. Reads that protect a version number or state transition may use FOR UPDATE.

7 Automatic Scheduling Backend

Separation of responsibilities

Service	Responsibility
SchedulingValidator	Builds the selected scope, checks curriculum, labs, priorities, dates, shifts, capacity and gaps, then stores a rule snapshot and conflicts.
AutomaticScheduler	Consumes a ready run, expands curriculum by batch, applies hard and soft constraints, creates exams and cohort rows and records unscheduled items.
ScheduleReviewService	Locks and unlocks papers, validates moves and prepares regeneration.
SchedulePublicationService	Performs final checks and controls approval and publication.

Scheduling algorithm

1. Lock and load a ready scheduling run and confirm the exam cycle is still draft.

2. Read the immutable rule snapshot and selected programme-course IDs.

3. Expand generic curriculum into the selected programme batches and their current semesters.

4. Exclude inactive, duplicate, locked or practical papers.

5. Load enabled dates and ordered shifts.

6. Build cohort occupancy from existing preserved papers.

7. Group common course IDs so matching papers can share the same slot.

8. Try preferred gap and daily rules first, then relax only soft preferences.

9. Never violate cohort slot collision or paper-duration constraints.

10. Create examinations, examination cohorts, run items, conflicts and audit data in one transaction.

Why the algorithm is flexible

The number of papers is not hard-coded. Capacity is derived from enabled dates and configured shifts. Preferred gaps are preserved when possible but relaxed on consecutive working days when necessary, ensuring realistic schedules without treating an example of nine or ten papers as a system limit.

8 Seating Invigilation and Attendance

Seating algorithm

1. Verify the date is enabled and belongs to the selected cycle and shift.

2. Reject unresolved pending paper registrations.

3. Load eligible students and reject same-session double examinations.

4. Load active rooms and only unblocked seats within usable capacity.

5. Group students by examination and interleave them round-robin.

6. Create the next locked version number for that session.

7. Write assignments and a separate unallocated list in one transaction.

Manual correction and publication

Only draft allocations can be corrected. The backend checks that a replacement seat is enabled, belongs to an active room and is unused. Manual overrides and reasons are stored and audited. Publication is blocked until seating validation passes.

Invigilation and attendance

Invigilation allocation respects faculty availability and uniqueness constraints. Replacement requests preserve requester, proposed faculty, reviewer and decision. Attendance updates are protected by CSRF and unique examination-student records, with correction identity retained.

9 Bulk Import and Export

Import pipeline

```
Upload -> store temporary file -> read rows -> normalize headers
       -> validate each row -> stage preview and errors
       -> user confirms -> transaction commit -> audit result
```

Import service	Key validation
StudentImportService	Roll-number pattern, programme, batch, session, semester and student fields
FacultyImportService	Employee identity and school or department references
ProgrammeImportService	School code, programme code, level and duration
CourseImportService	Programme, exact batch label, semester, course details, status and priority
RoomImportService	Room geometry, capacity and blocked seat coordinates
DateSheetImportService	Cycle date, shift, programme, batch, semester, section and written-paper policy

Export responses

CSV is used for portable operational lists. XLSX is used for formatted official date sheets. Response sanitizes attachment filenames and sets exact MIME and content headers so the browser downloads a valid file rather than rendering binary output as text.

10 Error Handling Audit and Deletion Safety

Error strategy

- Expected business errors throw RuntimeException with an understandable message.
- POST handlers roll back transactions and redirect with a flash message.
- Live-preview endpoints return JSON with status 422 for invalid requests.

- Unhandled production failures render a neutral 500 page.
- The health endpoint isolates database availability from page rendering.

Audit strategy

Sensitive actions record user ID, action code, entity type, entity ID and JSON before or after values. Important examples include schedule validation, generation, manual moves, approval, publication, seating correction and deletion.

Deletion strategy

Deletion type	Protection
Simple master	Permission, POST, CSRF and foreign-key restriction
Course mapping	Dedicated service checks historical scheduling dependencies
Published date sheet	Explicit workflow and dependent run-item handling
Exam cycle	Preview, two-step confirmation, dependency fingerprint and transaction

11 Testing Strategy

The suite contains focused logic tests and temporary-database integration tests. Database tests replace the production database name, create an isolated schema, load seed data, exercise services and drop the test database in a finally block.

Test	What it protects
RollNumberParserTest	Institutional roll-number rules
RolePolicyTest	Role permission boundaries
BatchProgressionTest	Current semester and completed-batch logic
WrittenPaperPolicyTest	Lab and practical exclusion
XlsxExporterTest	Workbook validity and export content
SeedTest	Fresh install and repeatable seed
Import tests	Staging, row validation, aliases and transactional commit
AutomaticSchedulingTest	Priority, generation and persisted run output
PreferredGapSchedulingTest	Gap preference and flexible fallback
SoictSchedulingTest	School and branch isolation
Deletion tests	Historical dependency safety
EndToEndTest	Cross-module examination workflow

Recommended test command

```
C:\xampp\php\php.exe -d zend.assertions=1 -d assert.exception=1
tests\AutomaticSchedulingTest.php
```

12 Deployment Performance and Maintenance

Deployment checklist

1. Upload the project with public as the served entry directory or retain the documented public URL.

2. Set APP_ENV production, APP_DEBUG false and the exact HTTPS APP_URL.

3. Copy hosting database credentials exactly; remote shared-host database hostname is not localhost.

4. Import schema.sql and then seed.sql for a fresh database.

5. Ensure storage upload and session locations are writable and not public.

6. Verify rewrite rules, assets, /health, login and one protected route.

7. Run a staging import and export before production use.

Performance practices

- Filter and join on indexed IDs, dates, shifts, programme, batch and semester.
- Select only necessary columns for reports and paginate growing lists.
- Keep expensive generation work transactional but avoid holding locks during user review.
- Use EXPLAIN for slow queries before adding indexes.
- Move very large exports or imports to queued jobs if the deployment grows beyond shared-host limits.

Backend improvement roadmap

- Extract controllers and a declarative router from the central Application class.
- Introduce versioned migrations for existing production databases.
- Add reusable request validation and structured application logging.
- Add automated full-suite execution in continuous integration.
- Add background jobs for large imports, exports and notification workflows.

13 Backend Jury Question Bank

Answer the first sentence directly. Continue with the implementation detail only when the jury asks how it works.

Q1. What backend architecture does the application use?

It uses a lightweight server-rendered PHP architecture with a front controller, custom routing, service classes, PDO repositories or queries, PHP views and a MySQL database. It follows MVC and service-layer principles without depending on a large framework.

Q2. Why did you use a front controller?

Apache rewrites non-file requests to public/index.php, giving one controlled entry point for bootstrapping, authentication, authorization, exception handling and response generation.

Q3. Explain the request lifecycle.

The browser reaches public/index.php, bootstrap loads configuration and services, Request captures method and path, Application dispatches the route, a handler calls domain services and PDO, View renders HTML or Response emits JSON, CSV, XLSX, redirect or an error status.

Q4. Why is public the document root?

Only public assets and index.php should be web-accessible. Application code, configuration, schema, storage and tests remain outside the web root, reducing accidental disclosure.

Q5. How does routing work without a framework?

Application dispatch compares normalized paths and HTTP methods. Dynamic identifiers use anchored regular expressions, while fixed GET pages use a final match expression.

Q6. What are the limitations of the current router?

It is centralized and easy to understand, but Application is large. A future refactor could move route definitions and controllers into separate modules while retaining the same service layer.

Q7. How is dependency creation handled?

Application boot creates the database, session and authentication objects. Domain services receive PDO through constructor injection, which makes their dependencies explicit and testable.

Q8. How is autoloading implemented?

bootstrap/app.php registers an SPL autoloader for the App namespace and maps class names to files under app using PSR-4-like directory conventions.

Q9. How are environment values loaded?

Environment reads non-empty KEY=VALUE lines from .env and places them in environment arrays. Configuration files use the env helper with safe defaults.

Q10. How is the database connection implemented?

Database lazily creates one PDO connection using utf8mb4, exception mode, associative fetching and native prepared statements with emulation disabled.

Q11. Why use PDO prepared statements?

Prepared statements separate SQL structure from user data, reduce SQL injection risk and provide consistent parameter binding and exception handling.

Q12. How is authentication implemented?

Auth selects the user and role by username, requires active status, verifies the stored password hash, regenerates the session ID, stores a safe user payload and updates last_login_at.

Q13. Why regenerate the session after login?

It prevents session fixation by replacing the pre-authentication session identifier after credentials are verified.

Q14. How are passwords protected?

The backend stores password_hash output and checks it with password_verify. Plain-text passwords are never used as database credentials or retained after login.

Q15. How is brute-force login reduced?

The session tracks failed attempts. Five failures trigger a sixty-second pause before additional sign-in attempts.

Q16. How are sessions secured?

Cookies use HttpOnly, SameSite Lax and Secure when HTTPS is active. The backend enforces inactivity timeout, regenerates after authentication and destroys the session on logout.

Q17. What is CSRF and how is it prevented?

CSRF tricks a signed-in browser into submitting an unwanted action. Every state-changing form includes a random session token and handlers validate it with hash_equals.

Q18. How does role-based authorization work?

RolePolicy maps role codes to permissions. Before dispatching a protected route, Application derives the required permission from the path and returns HTTP 403 when Auth cannot grant it.

Q19. Authentication versus authorization?

Authentication proves who the user is. Authorization decides what that authenticated user may do. The project handles them through Auth and RolePolicy respectively.

Q20. How is output escaping handled?

The e helper applies htmlspecialchars with quote and invalid-byte protection. Views should use it for all untrusted values to prevent stored or reflected XSS.

Q21. How are errors handled?

Application catches unhandled Throwables. Debug mode shows escaped details for development; production renders a generic 500 page so internal SQL, paths and secrets are not disclosed.

Q22. How are HTTP status codes used?

Successful pages use 200, form redirects use 302, validation JSON uses 422, forbidden routes use 403, missing routes use 404 and unexpected failures use 500.

Q23. How do flash messages work?

The session stores a message under a temporary flash namespace. The next request pulls and removes it, which fits the POST-Redirect-GET pattern.

Q24. What is POST Redirect GET?

After a successful or failed form submission the handler redirects to a GET page. This prevents accidental duplicate form submissions during refresh.

Q25. How is validation organized?

Simple request checks occur in handlers, reusable domain rules live in services or policy classes, and database constraints provide the final integrity boundary.

Q26. Why are services used?

Services isolate business operations such as scheduling, seating, imports and publication from HTTP rendering. This improves reuse, transaction control and focused testing.

Q27. How does bulk import work?

The upload is stored temporarily, TabularReader reads CSV or XLSX, a specific import service normalizes headers and validates every row, staged data and errors are previewed, and only a confirmed commit writes valid records.

Q28. Why split import into stage and commit?

Administrators can review invalid rows before changing master data. It also creates an audit-friendly boundary and lets the commit run in one transaction.

Q29. How are uploaded files protected?

The backend validates expected extension and structure, generates controlled storage names, avoids executing uploaded content and does not trust the original filename as a filesystem path.

Q30. How is automatic scheduling divided?

SchedulingValidator produces explainable readiness checks and a saved rule snapshot. AutomaticScheduler consumes only a ready run and creates the draft transactionally. Review and publication are separate services.

Q31. What are hard and soft scheduling constraints?

Hard constraints include enabled date and shift, one cohort per slot, written-paper eligibility and duration. Preferred gaps and daily paper limits are soft: they can be relaxed progressively when the calendar has insufficient spare slots.

Q32. How are branches kept separate?

Every task carries programme, batch, semester and section. Examinations may share a physical date and shift, but examination_cohorts preserves separate branch rows and conflict checks use a cohort key.

Q33. How is subject selection preserved?

The automatic scope is stored in the session and submitted programme_course IDs form an explicit whitelist. Filtering the UI does not redefine or reselect that whitelist.

Q34. How are labs excluded?

WrittenPaperPolicy is applied during validation, generation and date-sheet import so lab and practical subjects do not enter written scheduling through another path.

Q35. Why are scheduling operations transactional?

A generated schedule updates runs, examinations, cohorts, run items, conflicts and audit logs. A transaction ensures partial output is rolled back if any write fails.

Q36. How does manual move validation work?

ScheduleReviewService loads the paper and rule snapshot, verifies the target calendar date and shift, checks cohort collisions and duration, then updates and audits only when every hard rule passes.

Q37. How does regeneration preserve work?

Locked papers remain fixed. A new run is prepared from the previous scope, unlocked draft output is regenerated and shared-paper scope is checked to prevent accidental cross-cohort deletion.

Q38. How is publication controlled?

SchedulePublicationService performs final checks, requires an approved state and records approver and publisher identity before the date sheet becomes available to downstream seating operations.

Q39. How does seating generation work?

SeatingAllocator loads eligible students, rejects students with two papers in one session, loads enabled seats within room capacity, interleaves examination groups round-robin, creates a versioned allocation and stores unallocated students.

Q40. Why use round-robin seating?

Interleaving different examination groups reduces adjacent students from the same paper while keeping allocation deterministic and understandable.

Q41. How is seating validated?

The validator checks unresolved students, capacity, inactive rooms, duplicate students, duplicate seats, blocked seats and ineligible students before publication.

Q42. How are deletions made safe?

Simple masters rely on authorization, CSRF and foreign keys. Courses have dependency-aware deletion. Exam cycles require a preview, a short-lived fingerprint and typed confirmation before a transaction removes dependants.

Q43. Why use a deletion fingerprint?

It proves that the confirmed dependency summary matches the current cycle state, preventing deletion when records changed between preview and confirmation.

Q44. How are CSV responses generated?

Response sets text/csv and a sanitized attachment filename. Queries use explicit selected columns and application code serializes rows with proper CSV escaping.

Q45. How is XLSX generated?

XlsxExporter creates the Office Open XML package directly and Response returns the official XLSX MIME type, attachment name and exact content length.

Q46. How is the health endpoint useful?

GET /health reports application identity, PHP version, database connectivity and server time. It verifies deployment wiring without exposing credentials.

Q47. What security headers are configured?

Apache adds nosniff, SAMEORIGIN framing protection and strict-origin-when-cross-origin referrer policy. Hidden environment and Git files are denied.

Q48. How does the system work on InfinityFree?

The same PHP code runs under Apache-compatible rewriting, but APP_URL and provider database credentials must match hosting. Writable upload and session paths and shared-host SQL limitations must be respected.

Q49. How is the backend tested?

The tests cover roll parsing, roles, batch progression, written-paper policy, XLSX export, seed repeatability, imports, end-to-end flows, scheduling, preferred gaps, school isolation and safe deletion.

Q50. Unit test versus integration test in this project?

Pure logic tests such as role policy are unit-like. Tests that create a temporary MySQL database, import schema and seed, then execute services are integration tests.

Q51. Why is there no external framework?

The application has a small deployment footprint and explicit control flow suitable for shared hosting. The trade-off is more custom infrastructure code and a larger central Application class.

Q52. What would you refactor first?

I would extract route definitions and domain controllers from Application, introduce reusable request validators and migrations, then preserve existing services and tests as behavioural protection.

Q53. How would you scale the backend?

First add query profiling, indexes, caching for stable masters and background jobs for large exports. At larger scale, split stateless web instances from shared database, session and job infrastructure.

Q54. What is the strongest backend design decision?

The strongest decision is the staged workflow: validate before generation, review before publication and preserve audit records. It makes complex examination automation explainable and recoverable.

Appendix A Route and Module Demonstration

Demonstration	Backend path	What to explain
Health	GET /health	JSON response and database availability
Login	POST /login	CSRF, throttle, password verification and session regeneration
Course import	POST /courses/import	Stage, preview, validation and transaction commit
Schedule preview	POST /date-sheets/{cycle}/automatic/preview	Real-time non-persistent validation
Save validation	POST /date-sheets/{cycle}/automatic/validate	Persisted rule snapshot and conflicts
Generate	POST /date-sheets/{cycle}/automatic/generate	Transactional scheduling algorithm
Move paper	POST /automatic/runs/{run}/papers/{paper}/move	Hard-constraint validation and audit
Export	GET /date-sheets/{cycle}/export.xlsx	Binary response and valid workbook
Generate seating	POST /seating/generate	Eligibility, room capacity and round-robin placement
Delete cycle	GET and POST /exam-cycles/{id}/delete/confirm	Preview, fingerprint and typed confirmation

Appendix B Presentation Script and Checklist

Two minute backend explanation

Our backend is a custom strict-typed PHP application using a front controller and service-layer design. Apache sends application routes to public/index.php, bootstrap loads configuration and autoloading, and the Application dispatcher performs authentication and role permission checks before calling a handler. Database access uses PDO with native prepared statements and transactions. Reusable services handle imports, automatic scheduling, schedule review, publication, seating and safe deletion. The scheduler first stores a validated rule snapshot, then generates branch and batch aware papers while keeping hard constraints separate from preferred gaps. Seating uses eligible students and enabled seats, interleaves paper groups and creates versioned allocations. Security includes password hashing, session regeneration, timeout, CSRF tokens, output escaping, login throttling and production-safe errors. Tests cover pure policies and complete database-backed workflows.

Before the jury

- [ ] I can draw and explain the request lifecycle from browser to database and back.
- [ ] I can distinguish authentication, authorization, validation and auditing.
- [ ] I can explain prepared statements, CSRF, sessions and output escaping.
- [ ] I can explain why imports use stage, preview and commit.
- [ ] I can explain validator, scheduler, review and publication as separate services.
- [ ] I can explain hard constraints, soft preferences and transaction rollback.
- [ ] I can demonstrate /health, login, scheduling, seating and an export.
- [ ] I can state current limitations and a realistic refactoring roadmap.